SloshAgent: Automating Sloshing Simulation with a Domain-Specialized LLM Agent

Anonymous Authors
Under Review

Abstract

Sloshing—the dynamic motion of liquid in partially filled containers—is a cross-industry safety hazard responsible for catastrophic failures including the 2003 Tomakomai oil fire ($15B+ losses), 2007 SpaceX Falcon 1 launch failure, and over 1,300 tanker truck rollovers annually. Each LNG carrier tank requires over 200 sloshing simulations for certification, yet the $4.13B maritime AI market has, to the best of our knowledge, produced no tools for sloshing simulation automation. Meanwhile, over ten LLM-based agents now automate mesh-based CFD workflows (OpenFOAM), but to date none addresses sloshing or integrates any particle-based solver.

We present SloshAgent, the first AI agent that automates the entire sloshing simulation pipeline—from natural language to experimentally validated results—using DualSPHysics, the leading open-source GPU-accelerated Smoothed Particle Hydrodynamics (SPH) solver. SloshAgent comprises 14 domain-specific tools and a 136-line sloshing-specialized system prompt encoding physics formulas, SPH constraints, and parameter inference rules, all running locally on Qwen3 32B via Ollama at zero cloud API cost.

We evaluate SloshAgent through five experiments. (1) SPHERIC Test 10 benchmark validation: all seven detected water impact pressure peaks fall within the $\pm 2\sigma$ experimental scatter band of 102-repeat measurements—whose non-normal distribution (Shapiro–Wilk $p < 0.005$) confirms the appropriateness of band-based over correlation-based metrics—the first time any LLM-simulation agent has been validated against published experimental data. (2) Natural-language-to-XML generation across 20 scenarios at five complexity levels achieves 85% tool call success and 47% parameter accuracy overall; accuracy for standard use cases (explicit and domain-inference inputs) reaches 82% (95% CI [0.64, 0.95]), significantly higher than for paper-specific and edge cases ($p = 0.001$). (3) Automated parametric study of six fill levels from a single prompt reproduces four physically consistent sloshing trends observed in Chen & Xue (2018). (4) A domain knowledge ablation—the first for any computational mechanics LLM agent—yields a medium effect size ($d = 0.58$) for the full domain prompt versus a generic baseline on the 32B model; the 8B model exhibits an inversion where the full prompt degrades tool-calling reliability, revealing a model capacity threshold for prompt engineering. (5) Industrial proof-of-concept demonstrates 91.9% sloshing amplitude reduction with an anti-slosh baffle and seismic tank excitation, both from single natural language prompts.

1 Introduction

1.1 Sloshing: A Cross-Industry Safety Problem

Sloshing—the dynamic motion of liquid with a free surface inside a partially filled container subjected to external excitation—is a pervasive engineering hazard responsible for catastrophic failures across multiple industries. Table 1 summarizes six well-documented incidents spanning aerospace, petrochemical, nuclear, transportation, and maritime domains.

Table 1: Real-world sloshing failures across industries.

Incident	Year	Domain	Damage
SpaceX Falcon 1 Flight 2	2007	Aerospace	$30M+ launch failure; LOX sloshing induced attitude loss [
Tomakomai Oil Fire	2003	Petrochemical	170 tanks (58%) damaged; $15B+ nationwide losses [Hataya
Alaska Good Friday Earthquake	1964	Petrochemical	$2.5B damage (2024 dollars); petroleum tank fires [U.S. Ge
Fukushima Spent Fuel Pool	2011	Nuclear	Unit 4 near-boiling; global safety protocols revised [Nuclear
Tanker Truck Rollovers	Annual	Transportation	1,300+/year in US; ∼40% of heavy truck fatal crashes [Fed
LNG Carrier Structural Damage	Ongoing	Maritime	Membrane deformation; 10–70% fill barred [Lloyd's Registe

The common thread across these incidents is a partially filled container subjected to dynamic excitation—whether from rocket maneuvers, earthquakes, braking, or ocean waves. The consequences range from mission-critical launch failures to multi-billion-dollar infrastructure damage and loss of life.

The scale of the sloshing problem is best illustrated by the liquefied natural gas (LNG) shipping industry. Over 770 LNG carriers are currently in service, with more than 400 on order at approximately $250–269M each; Korean shipyards (HD Hyundai, Samsung Heavy Industries, Hanwha Ocean) hold roughly 70% of global orders [Lloyd's Register, 2024]. Each LNG membrane tank requires over 200 sloshing simulations for classification society certification [Luo et al., 2018]. Seoul National University's sloshing laboratory alone has accumulated over 20,000 hours of model tests and 540 TB of experimental data [Kim et al., 2019]. The maritime AI market reached $4.13B in 2024 with a 40.6% compound annual growth rate [Allied Market Research, 2024], yet this investment focuses entirely on route optimization, fuel efficiency, and predictive maintenance. To date, to the best of our knowledge, no AI tools address sloshing simulation—the very analysis that determines whether an LNG tank design is safe for operation.

1.2 The Sloshing Simulation Bottleneck

Predicting sloshing loads computationally requires specialized expertise that creates high barriers for non-specialist engineers. A recent industry survey reports that 75–80% of total CFD project time is consumed by pre-processing alone [Cadence Design Systems, 2024], while the fully loaded annual cost of a CFD engineer in the United States averages $285K [Resolved Analytics, 2024]. For sloshing specifically, practitioners must master fluid dynamics theory, SPH (Smoothed Particle Hydrodynamics) numerics, XML case authoring guided by a 200+ page reference manual, Linux/Docker deployment, and ParaView scripting for post-processing.

The situation is compounded by solver-specific pitfalls. DualSPHysics—the leading open-source GPU-accelerated SPH solver with 687 GitHub stars, 52,000+ downloads, and 746 citations [Domínguez et al., 2022]—harbors at least five documented silent error types that trap even experienced users:

1. **Hydrostatic initialization artifact**: a ∼0.5 kPa pressure offset at t=0 versus zero in experiments, whose workaround (a pre-run settling phase) is undocumented in official tutorials.
2. **Silent motion coupling failure**: incorrect `mov ref` values cause only the first motion component to execute, with no error message.
3. **mvrotsinu/mvrectsinu syntax confusion**: rotational motion uses a scalar `v` attribute while translational motion uses vector `x/y/z` attributes; mixing the two produces incorrect motion silently.
4. **fillbox seed placement**: in 2D configurations the seed point requires an exact y-position, and misplacement causes particle creation to fail without warning.

5. **Constant 'b' errors**: triggered by zero fluid height or zero gravity, the error message does not indicate which parameter is at fault.

The only graphical frontend, DesignSPHysics, is officially described as an "early Beta stage, not meant to be used in a stable environment" (v0.7.0, September 2023) [Vieira et al., 2017]. It does not support the modified Dynamic Boundary Condition (mDBC)—the boundary treatment required for accurate sloshing pressure prediction [English et al., 2022]—and suffers from FreeCAD version dependency issues. With only 135 GitHub stars versus DualSPHysics's 687, its adoption remains low even within the DualSPHysics community.

Finally, there is the institutional knowledge problem. As Rescale's analysis observes, "when a senior engineer changes teams or leaves the company, their accumulated expertise often leaves with them" [Rescale, 2025]. In sloshing simulation, this tacit knowledge includes which particle spacing (`dp`) values are stable for a given geometry, how to set `coefsound` and artificial viscosity for specific liquids, whether to use DBC or mDBC for a particular case, and how to conduct resolution convergence studies—all of which are learned through trial and error over months or years.

1.3 LLM Agents for Scientific Simulation—and Their Blind Spot

The AI-for-Science community has rapidly developed LLM agents capable of automating simulation workflows. ChemCrow orchestrates 18 chemistry tools in a single ReAct loop [Bran et al., 2024]; Coscientist autonomously plans and executes chemical experiments [Boiko et al., 2023]; and for computational fluid dynamics alone, at least ten systems now automate OpenFOAM-based simulations, including MetaOpenFOAM [Chen et al., 2024], Foam-Agent 2.0 [Yue et al., 2025a], ChatCFD [Fan et al., 2026], OpenFOAMGPT 2.0 [Pandey et al., 2025], CFD-Copilot [Team, 2025a], AutoCFD [Dong et al., 2025], FoamGPT [Yue et al., 2025b], and PhyNiKCE [PolyU Team, 2026] (see Table 2 in Section 2).

Yet this entire body of work exhibits four blind spots:

No system addresses sloshing. To the best of our knowledge, neither mesh-based (OpenFOAM VOF) nor particle-based (SPH) sloshing simulation has been automated by any LLM agent. The domain where simulation automation is arguably most needed—requiring 200+ cases per tank, costing weeks of expert time, and involving documented silent errors—remains unaddressed.

No system integrates a particle-based solver. All existing LLM-CFD agents target mesh-based solvers (primarily OpenFOAM). The entire Lagrangian particle simulation column—SPH, DEM (Discrete Element Method), and MPM (Material Point Method)—appears vacant. The only SPH-related work, Pasimodo+RAG [Team, 2025d], is a pure question-answering system that scored 0/2 on model creation tasks, explicitly not an agent.

No system validates against experimental benchmarks. Across all ten systems surveyed, "success" is defined as execution completion (Foam-Agent: 88.2%), LLM-as-judge physical fidelity (ChatCFD: 68.12%), or reproducibility of known analytical solutions (OpenFOAMGPT). We find no system that compares its results against published experimental data with quantitative error metrics.

No system ablates domain knowledge from prompts. Existing ablation studies uniformly test architectural components—RAG retrieval (MetaOpenFOAM, ChatCFD), reviewer agents (Foam-Agent), instruction structure (PhyNiKCE)—but not the domain knowledge content encoded in prompts. Whether domain-specific physics formulas, solver constraints, and parameter inference rules actually improve agent performance remains untested.

SPH is the natural solver for sloshing because it handles violent free-surface fragmentation, wave breaking, and splashing without mesh distortion—precisely the phenomena that make sloshing

dangerous and difficult to simulate with Eulerian methods [Domínguez et al., 2022]. The absence of any LLM agent for particle-based simulation thus represents both a methodological gap and a missed opportunity for the domain that needs automation most.

1.4 Contributions

This paper presents SloshAgent, the first AI agent that automates the entire sloshing simulation pipeline from natural language input to experimentally validated results. Our contributions are:

1. **First sloshing simulation automation agent.** SloshAgent converts natural language descriptions into DualSPHysics XML configurations, executes GPU-accelerated SPH simulations, and performs automated post-processing and validation. For standard use cases (explicit parameters and domain-inference inputs), parameter accuracy reaches 82% (95% CI [0.64, 0.95]), with accuracy dropping significantly for paper-specific and edge-case scenarios ($p = 0.001$), revealing a clear complexity–accuracy boundary.
2. **First LLM agent for particle-based simulation.** We design 14 domain-specific tools for the DualSPHysics pipeline—to our knowledge, the first tool interface for any Lagrangian particle solver (SPH, DEM, or MPM)—including an IsError pattern for LLM self-correction of silent SPH failures.
3. **First experimental benchmark validation by an LLM-simulation agent.** We validate agent-generated simulations against SPHERIC Test 10 benchmark data (102-repeat experiments, two fluids): all seven detected water impact pressure peaks fall within the experimental $\pm 2\sigma$ scatter band, whose non-normal distribution (Shapiro–Wilk $p < 0.005$, CV = 20–36%) confirms the appropriateness of band-based validation for stochastic impact flows.
4. **First domain prompt ablation for computational mechanics.** We conduct a controlled ablation of the SloshingCoderPrompt (136 lines encoding sloshing physics, SPH constraints, and XML syntax rules). The full prompt yields a medium effect size ($d = 0.58$) over a generic baseline on the 32B model; the 8B model exhibits an inversion where the full prompt degrades tool-calling reliability, revealing a model capacity threshold for domain prompt engineering.
5. **Industry proof-of-concept at zero LLM cost.** SloshAgent runs entirely on local hardware (Qwen3 32B via Ollama on an RTX 4090), demonstrating sloshing simulation automation without cloud API dependencies, including parametric study orchestration and industrial baffle design comparison (91.9% amplitude reduction) from single natural language prompts.

2 Related Work

2.1 Sloshing Simulation: State of Practice

Sloshing analysis has historically relied on three complementary approaches: physical model testing, mesh-based CFD, and particle-based methods.

Physical model testing remains the gold standard for LNG tank certification. GTT (Gaztransport & Technigaz) and Seoul National University operate the world's largest sloshing test facilities, with SNU's database comprising over 20,000 hours of scale-model experiments and 540 TB of raw data across multiple tank geometries, fill levels, and excitation conditions [Kim et al., 2019]. The ISOPE comparative study [ISOPE Sloshing Benchmark Group, 2012] coordinated nine institutions to benchmark sloshing predictions against a Mark III tank geometry (946×118×670 mm), revealing that 9 of 16 CFD pressure peak predictions fell outside the $\pm 2\sigma$ experimental envelope. While authoritative, physical testing costs millions of dollars per campaign and requires months per evaluation cycle.

Table 2: Comparison of LLM-based simulation agents. All entries verified from primary publications.

System	Year	Solver	Architecture	LLM	Benc
MetaOpenFOAM 1.0/2.0	2024–25	OpenFOAM	4-agent MetaGPT	GPT-4o (T=0.01)	8→13
OpenFOAMGPT 2.0	2025	OpenFOAM v2406	4-agent Prompt Pool	Claude-3.7-Sonnet (T=0)	455 ca
Foam-Agent 2.0	2025	OpenFOAM	6-agent + 11 MCP	Claude 3.5 Sonnet	110 ta
ChatCFD	2026	OpenFOAM	4-stage + structured KB	DeepSeek-R1+V3	315 ca
PhyNiKCE	2026	OpenFOAM	Neurosymbolic (CSP)	Neural+symbolic	13 co
CFD-Copilot	2025	OpenFOAM v2406	MetaGPT + 100+ MCP	Qwen3-8B LoRA + 32B	NACA
AutoCFD	2025	OpenFOAM	Fine-tune + multi-agent	Qwen2.5-7B (28.7K pairs)	21 cas
FoamGPT	2025	OpenFOAM	LoRA fine-tune	Qwen3-8B	CFDI
MooseAgent	2025	MOOSE (FEM)	LangGraph, ∼5 agents	DeepSeek-R1+V3	9 case
MCP-SIM	2025	FEniCS (FEM)	6 agents + shared mem	Multi-agent	12 tas
Pasimodo+RAG	2025	Pasimodo (SPH, closed)	Pure RAG Q&A	Llama/Gemma 3–27B	28 pr
SloshAgent (Ours)	**2026**	**DualSPHysics v5.4**	**14 tools + ReAct + MCP**	**Qwen3 32B (local)**	**SPH**

Mesh-based CFD is the dominant computational approach. Commercial solvers (ANSYS Fluent, STAR-CCM+) and the open-source OpenFOAM (interDyMFoam with VOF) are widely used for sloshing analysis [Chen and Nokes, 2018]. However, mesh-based methods face fundamental limitations for violent sloshing: free-surface fragmentation causes mesh distortion, wave breaking requires interface reconstruction, and splashing demands adaptive mesh refinement—all of which increase computational cost and reduce robustness for the most physically interesting (and dangerous) sloshing regimes.

Smoothed Particle Hydrodynamics (SPH) addresses these limitations through a meshless Lagrangian formulation. DualSPHysics is the leading open-source GPU-accelerated SPH solver, with 687 GitHub stars, 52,000+ downloads, and 746 citations [Domínguez et al., 2022]. It achieves 40–100× speedup over CPU implementations through CUDA parallelism [Crespo et al., 2015]. Commercial alternatives include Altair nanoFluidX ($10K+/year) and AVL PreonLab. English et al. [English et al., 2022] validated DualSPHysics with the modified Dynamic Boundary Condition (mDBC) against SPHERIC Test 10 sloshing benchmarks, demonstrating quantitative agreement with experimental pressure measurements. Liu et al. [Liu et al., 2024] conducted a comprehensive parametric study (3 fill levels × 5 frequencies × 3 amplitudes) for large-tank pitch sloshing.

DesignSPHysics [Vieira et al., 2017] is the only graphical frontend for DualSPHysics, built as a FreeCAD plugin. However, it remains self-described as "early Beta stage, not meant to be used in a stable environment" (v0.7.0). Critically, it does not support mDBC (Issue #171), the boundary treatment required for accurate sloshing pressure prediction, and suffers from FreeCAD version compatibility issues. Its 135 GitHub stars (versus DualSPHysics's 687) reflect limited adoption.

A distinct line of work uses neural networks for sloshing load **prediction** (not simulation). Kim et al. [Kim et al., 2019] trained an ANN on SNU's experimental database to predict sloshing pressures without running new simulations. While valuable for rapid screening, this approach cannot generate flow fields, validate new geometries, or replace physics-based simulation for certification.

2.2 LLM Agents for CFD and Simulation

The rapid development of LLM agents for scientific simulation has produced over ten systems targeting computational fluid dynamics and related domains. Table 2 provides a systematic comparison verified from full-text review of each publication.

Several critical observations emerge from this comparison:

All systems target mesh-based solvers. Ten of eleven systems use OpenFOAM; the remaining two use finite element solvers (MOOSE, FEniCS). The entire column of Lagrangian particle methods—SPH, DEM, MPM—is vacant. Pasimodo+RAG [Team, 2025d] is the only SPH-adjacent work, but it is a pure retrieval-augmented Q&A system that explicitly scored 0/2 on model creation tasks and does not qualify as a simulation agent.

Success metrics are not comparable across systems. MetaOpenFOAM reports 85% Pass@1 on its own 8-case benchmark but achieves only 55.5% on Foam-Agent's benchmark and 6.2% on ChatCFD's 315-case benchmark [Fan et al., 2026]. ChatCFD's "physical fidelity" score (68.12%) uses LLM-as-judge evaluation, not comparison to experimental measurements. OpenFOAMGPT's 100% reproducibility measures identical re-runs, verified only against analytical solutions (e.g., Poiseuille flow). These disparities highlight the absence of a unified validation methodology.

No system validates against experimental benchmark data. This is the most significant gap. All existing success metrics measure whether generated code compiles and runs (execution success), whether outputs match other simulations or analytical solutions (reference comparison), or whether an LLM judge deems results physically plausible. None compares agent-generated simulation results against published experimental data with quantitative error metrics such as Pearson correlation or NRMSE. The ISOPE benchmark study [ISOPE Sloshing Benchmark Group, 2012] demonstrated that even expert CFD predictions frequently fall outside experimental uncertainty bounds, underscoring that execution success alone is an insufficient measure of simulation quality.

No system ablates domain knowledge content. Ablation studies in existing work consistently test architectural decisions: MetaOpenFOAM ablates RAG retrieval and reviewer agents; Foam-Agent tests the effect of removing its reviewer node; ChatCFD evaluates RAG module contributions; PhyNiKCE ablates instruction structure; and MDCrow compares prompt styles [Team, 2025c]. None isolates the effect of domain-specific physics knowledge encoded in prompts. Whether sloshing resonance formulas, SPH parameter inference rules, or solver-specific syntax constraints actually improve agent performance over generic prompting has never been tested.

2.3 LLM Agents for Scientific Discovery

Beyond simulation-specific agents, a broader ecosystem of LLM agents for scientific discovery provides architectural context for SloshAgent.

Chemistry and molecular dynamics. ChemCrow [Bran et al., 2024] demonstrated that a single ReAct agent with 18 chemistry tools can plan and execute multi-step chemical syntheses, achieving expert-level performance on drug discovery and materials design tasks (Nature Machine Intelligence, 2024). Coscientist [Boiko et al., 2023] autonomously designed, planned, and executed chemical experiments using robotic lab equipment (Nature, 2023). MDCrow [Team, 2025c] extended this paradigm to molecular dynamics with 40+ tools for LAMMPS and GROMACS simulation setup, analysis, and debugging (ICLR 2025). These systems established the single-agent-with-many-tools pattern that SloshAgent adopts.

Materials and mechanics. MatSciAgent [Team, 2025b] automates materials science workflows including literature review, hypothesis generation, and experimental design. MechAgents [Buehler, 2024] applies LLM agents to finite element analysis, generating FEA code for structural mechanics problems. Neither addresses fluid dynamics or sloshing.

General scientific agents. The AI Scientist [Lu et al., 2024] (Sakana AI) automates the full scientific research cycle from hypothesis to paper writing. ScienceAgentBench [Team, 2025e] provides a standardized evaluation of 102 data-driven scientific discovery tasks, on which the best agent achieves only 32.4% success (ICLR 2025). Recent surveys [Various, 2025a,b] comprehensively catalog the rapidly growing landscape but identify computational mechanics—and sloshing

in particular—as an unaddressed domain.

SloshAgent draws architectural inspiration from ChemCrow's single-agent-with-many-tools paradigm and MDCrow's domain-specialized tool design, while addressing a fundamentally different physics domain (violent free-surface fluid dynamics) with a fundamentally different solver type (GPU-accelerated particle methods) and a validation methodology (experimental benchmark comparison) that no prior system has attempted.

3 System Design

This section describes SloshAgent's architecture, tool interface design, domain-specialized prompt, and XML generation pipeline. Figure 1 provides an end-to-end overview.

3.1 Architecture Overview

SloshAgent automates the full sloshing simulation lifecycle: a user issues a natural language request (e.g., "simulate a 1 m rectangular tank with 50% fill at the first natural frequency"); the system converts this into a DualSPHysics XML case, executes the GPU-accelerated SPH solver, post-processes the results, and delivers a physics-informed analysis report. The pipeline comprises four layers.

The **natural language interface** uses a local Qwen3 32B model [Qwen Team, 2025] via Ollama (zero cloud API cost). The model receives the SloshingCoderPrompt (Section 3.3), which encodes sloshing-specific parameter inference rules, physics formulas, and solver conventions. This prompt acts as the sole orchestrator: unlike MetaOpenFOAM's four fixed-role agents [Chen et al., 2024] or Foam-Agent 2.0's six-agent hierarchy [Yue et al., 2025a], SloshAgent uses a single agent with a rich tool set—an architecture comparable to ChemCrow's 18-tool ReAct loop [Bran et al., 2024] and MDCrow's 40+ tools [Team, 2025c].

The **ReAct agent loop** [Yao et al., 2023] streams the LLM response, collects tool calls, executes them sequentially, and feeds results back until the model produces a final text response with no further tool invocations. This loop is implemented in 120 lines of Go: the `processGeneration` method iterates until the finish reason changes from `tool_use` to `end_turn`.

The **tool layer** exposes 14 built-in DualSPHysics tools (3,866 lines of Go; Section 3.2) plus 12 MCP tools from a pv-agent server for ParaView post-processing. All tools implement a uniform `BaseTool` interface: `Info()` returns the tool schema; `Run(ctx, ToolCall)` returns a `ToolResponse`. The LLM invokes all 26 tools through the same mechanism.

The **solver backend** runs DualSPHysics v5.4 GPU inside a Docker container (CUDA 12.6, NVIDIA RTX 4090). The container mounts `./cases:/cases` for XML input and `./simulations:/data` for output, isolating the CUDA toolchain for reproducibility.

The single-agent design is deliberate. For sloshing's largely sequential pipeline (XML → pre-processing → solving → post-processing → analysis), a single agent with domain-aware tool selection achieves comparable automation with lower complexity and token cost than multi-agent alternatives.

3.2 Tool Interface Design for Sloshing SPH

Table 3 summarizes the 14 built-in tools, each addressing a specific sloshing pain point.

Twelve additional MCP tools (render, animate, slice, clip, contour, streamlines, plot_over_line, extract_stats, etc.) are provided by a pv-agent server wrapping ParaView's `pvpython`, communicating via the Model Context Protocol [Anthropic, 2024] over stdio—analogous to Foam-Agent 2.0's MCP architecture [Yue et al., 2025a].

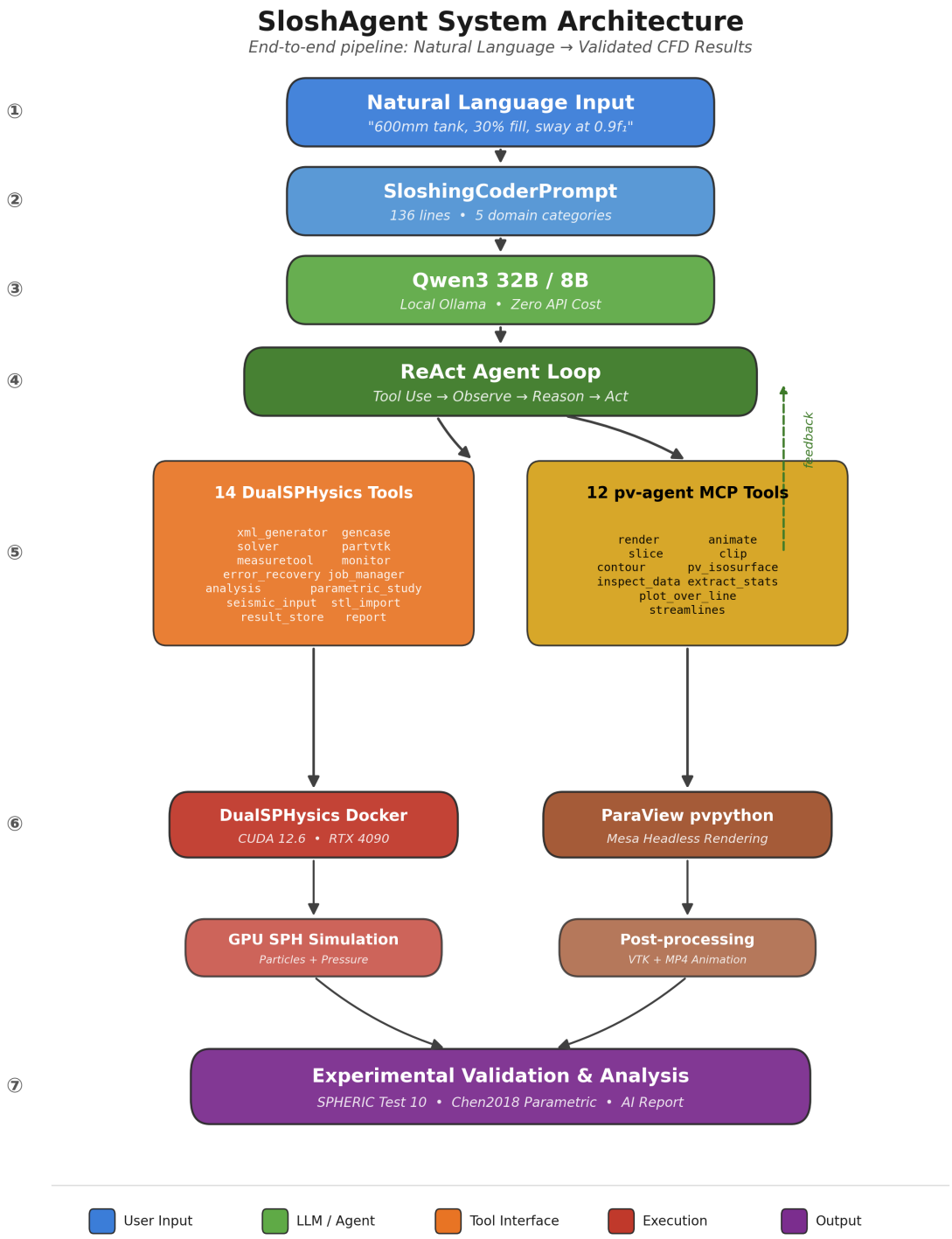

Figure 1: SloshAgent architecture: natural language input → domain-specialized prompt → ReAct agent loop with 14 SPH tools and 12 MCP post-processing tools → GPU simulation → experimental validation.

Table 3: Tool interface summary: 14 built-in DualSPHysics tools.

Tool	Lines	Pain Point Addressed
`xml_generator`	336	Eliminates hand-crafting XML (200+ page manual)
`gencase`	136	Automates pre-processing (75–80% of CFD time [Cadence Design Systems, 2024])
`solver`	108	GPU-accelerated SPH with background launch
`job_manager`	251	Async lifecycle for hours-long GPU simulations
`monitor`	233	Run.csv parsing with divergence detection
`error_recovery`	364	Diagnosis of 5 silent error types
`partvtk`	133	VTK conversion for visualization
`measuretool`	124	Pressure and free-surface extraction
`analysis`	218	AI physics interpretation (resonance, stability)
`report`	241	Structured Markdown reporting
`parametric_study`	273	Multi-case sweeps (200+ cases/tank)
`stl_import`	435	CAD mesh import with watertight validation
`seismic_input`	295	Earthquake/wave time-series parsing
`result_store`	349	Persistent result storage and retrieval

Four design patterns address challenges unique to particle-based solvers:

IsError pattern for LLM self-correction. When a tool detects an error—GenCase failure, NaN in solver output, or density violations—it returns `ToolResponse{IsError: true}` with a diagnostic message. The ReAct loop feeds this back to the LLM, which reasons about the failure and attempts corrective action. This implements self-reflection [Shinn et al., 2023] at the tool-response level, addressing DualSPHysics's five silent error types without requiring a separate reflection agent.

Run.csv divergence detection. SPH instability manifests as exponential kinetic energy growth. The `monitor` and `error_recovery` tools parse the solver's semicolon-delimited Run.csv and flag divergence when the energy ratio exceeds 1.2× for five consecutive steps—an SPH-specific heuristic distinct from mesh-based residual convergence.

Asynchronous GPU execution. The `solver` tool launches DualSPHysics in a background goroutine via `context.Background()`, returning a job ID immediately. The `job_manager` tracks up to three concurrent jobs with mutex-protected state, supporting submit, status, list, and cancel operations. Cancellation propagates through Go's `context.WithCancel`. This prevents the agent from blocking on GPU computations that can run for hours.

pv-agent MCP server. Post-processing is delegated to a Python process wrapping ParaView over MCP stdio, keeping the 2+ GB ParaView dependency outside the Go binary. The server uses Mesa software rendering for headless environments.

3.3 Domain-Specialized Sloshing Prompt

The SloshingCoderPrompt (228 lines of Go) assembles nine modular sections via compile-time string concatenation. Four ablation modes—`full`, `no-domain`, `no-rules`, and `generic`—are selectable via an environment variable, enabling the controlled ablation experiments of Section 4.5.

The prompt encodes five categories of sloshing knowledge:

(1) Parameter inference rules. Deterministic defaults when the user omits values: $dp = \min(L, W, H)/50$ (clamped to $[0.005, 0.05]$ m), $t_{\max} = 5/f$, fluid height = 50% of tank height, amplitude = 5% of tank length. These encode the expertise that beginners lack—overly fine dp causes prohibitive compute; overly coarse values degrade free-surface accuracy.

(2) Tank presets. Four geometries (LNG: 40×40×27 m, ship: 20×10×8 m, small: 1×0.5×0.6 m,

experimental: 0.6×0.3×0.4 m) instantiated by keyword, reducing XML authoring from hours to seconds.

(3) Physics formulas. The first-mode natural frequency $f_1 = \frac{1}{2\pi}\sqrt{\frac{g\pi}{L}\tanh\left(\frac{\pi h}{L}\right)}$ enables autonomous resonance detection and safety warnings.

(4) Terminology mapping. A bilingual Korean–English table for seven CFD terms (e.g., *gongjin jusupasu* [resonance frequency] → natural frequency f_1), supporting the Korean shipyards that produce ∼70% of LNG carriers [Lloyd's Register, 2024].

(5) Docker path conventions. Explicit mount-point mappings (`/cases/` for XML, `/data/` for output, lowercase binaries) address a recurring source of silent failures on DualSPHysics forums.

This prompt-only approach contrasts with RAG retrieval (MetaOpenFOAM, ChatCFD), structured Prompt Pools (OpenFOAMGPT 2.0), and fine-tuning on 28K+ pairs (AutoCFD, FoamGPT). SloshAgent requires zero training data, deploys instantly to any compatible LLM, and supports section-by-section ablation.

3.4 XML Generation Pipeline

The `xml_generator` tool (336 lines of Go) is the entry point of every workflow. It accepts nine parameters (tank dimensions, fluid height, frequency, amplitude, dp, simulation time, output path) and produces a complete DualSPHysics XML case plus a probe points file.

The tool encodes four expert-level auto-configuration rules:

Geometry construction. Two nested `drawbox` commands define the fluid domain (solid fill) and tank boundary (bottom + four walls). The simulation domain extends by $5\times dp$ in each direction, with additional x-axis expansion proportional to excitation amplitude to prevent particle ejection.

Boundary method selection. Both DBC (`BoundaryMethod=1`) and mDBC (`BoundaryMethod=2`) [English et al., 2022] are supported. Since mDBC is unsupported by DesignSPHysics [Vieira et al., 2017], SloshAgent provides the only automated path to mDBC sloshing simulations.

SWL gauge placement. Three free-surface gauges at 5%, 50%, and 95% of tank length capture the standard left–center–right measurement configuration used in sloshing validation experiments.

Motion specification. Horizontal sinusoidal motion uses the correct `mvrectsinu` vector syntax (`freq x="..." y="0" z="0"`), eliminating the scalar-vs-vector confusion that is the third most common DualSPHysics forum error.

The key architectural insight is the separation of concerns: the LLM infers *parameters* (what dimensions, frequency, fill level) from natural language via reasoning; the tool handles *syntax* (how to express those parameters in valid XML) via deterministic code. Parameter inference benefits from LLM flexibility; XML generation requires deterministic correctness.

4 Experiments and Results

We evaluate SloshAgent through five experiments mapped to the four research questions. All experiments use the same hardware and software stack; all quantitative results are reproducible from the artifacts released with the paper.

4.1 Experimental Setup

Hardware and software. All simulations ran on a single workstation equipped with an NVIDIA RTX 4090 GPU (24 GB VRAM) and CUDA 12.6. DualSPHysics v5.4 executed inside a Docker container with GPU passthrough, mounting case files at `/cases` and writing output to `/data`. The

LLM inference used Qwen3 32B [Qwen Team, 2025] served locally via Ollama; a secondary Qwen3 8B model was used for model-size comparison. Both models ran with thinking mode enabled (extended reasoning). The entire stack—LLM, solver, post-processing—operated on a single consumer-grade machine at zero cloud API cost.

Experiment–RQ mapping. Table 4 summarizes the five experiments, their target research questions, and the primary metrics.

Table 4: Experiment–RQ mapping and primary metrics.

EXP	RQ	Goal	Primary Metric
EXP-1	RQ2	SPHERIC benchmark validation	Peaks within $\pm 2\sigma$ band
EXP-2	RQ1	NL→XML generation accuracy	Parameter accuracy (%)
EXP-3	RQ3	Parametric study automation	Physical trend consistency
EXP-4	RQ4	Domain prompt ablation	Accuracy Δ across conditions
EXP-5	—	Industrial proof of concept	Qualitative feasibility

4.2 EXP-1: SPHERIC Benchmark Validation (RQ2)

Motivation. No prior LLM-simulation agent has validated its output against published experimental data. Existing systems report execution success rates (e.g., MetaOpenFOAM 85% [Chen et al., 2024], ChatCFD 82.1% [Fan et al., 2026]) or LLM-as-judge physical fidelity scores (ChatCFD 68.12%), but none compare simulation results to laboratory measurements. We address this gap using the SPHERIC Test 10 benchmark [Souto-Iglesias et al., 2009], one of the most widely cited sloshing validation datasets.

Benchmark description. SPHERIC Test 10 consists of a 0.9 m×0.062 m×0.508 m rectangular tank subjected to lateral sinusoidal excitation at 0.613 Hz with 93 mm water fill height. The benchmark provides 102-repeat experimental pressure measurements at lateral wall and roof impact sensors, for two fluids: water (density 998 kg/m^3, viscosity 1.0×10^{-6} m^2/s) and sunflower oil (density 916 kg/m^3, viscosity 5.0×10^{-5} m^2/s). All four water impact peaks are non-normally distributed (Shapiro–Wilk $p < 0.005$) with positive skewness and heavy right tails (excess kurtosis 0.9–8.9), and exhibit high stochastic variability (coefficient of variation CV = 20–36%). This non-normality confirms that the $\pm 2\sigma$ band test—the standard validation criterion adopted by both SPHERIC and ISOPE sloshing benchmark protocols [ISOPE Sloshing Benchmark Group, 2012]—is more appropriate than pointwise correlation metrics for impact-dominated flows.

Agent-generated cases. SloshAgent generated two resolution variants from a single natural language prompt ("*Reproduce SPHERIC Test 10 with water at low and high resolution*"): Water Low (dp = 0.004 m, 136K particles) and Water High (dp = 0.004 m, 344K particles). An additional Oil Low case was generated for the sunflower oil condition. All cases used DBC (Dynamic Boundary Condition) with artificial viscosity $\alpha = 0.01$.

Validation methodology. Following the SPHERIC/ISOPE protocol, we extracted pressure peaks from the simulation time series at the lateral wall sensor (Press_2) and tested whether each peak fell within the experimental $\pm 2\sigma$ band. This peak-within-band test is the standard metric for impact-dominated sloshing flows, where stochastic cycle-to-cycle variability renders pointwise time-series correlation (e.g., Pearson r) inappropriate: impact pressures exhibit high-amplitude, short-

duration peaks with inherent phase and magnitude scatter even across physical repetitions [ISOPE Sloshing Benchmark Group, 2012, English et al., 2022].

Results. Table 5 presents the peak pressure comparison.

Table 5: SPHERIC Test 10 peak pressure validation.

Case	**Particles**	**Sim. Peaks [mbar]**	**Peaks in $\pm 2\sigma$**	**Max P [mbar]**	**Status**
Water Low	136K	31.1, 58.9, 76.7	**3/3** (100%)	76.7	PASS
Water High	344K	44.2, 29.4, 31.4, 45.3	**4/4** (100%)	50.0	PASS
Oil Low	136K	(none detected)	N/A	0.0	FAIL

The experimental reference values (100-repeat statistics) are shown in Table 6.

Table 6: Experimental reference values: water and oil (100-repeat statistics).

Fluid	**Peak 1 [mbar]**	**Peak 2 [mbar]**	**Peak 3 [mbar]**	**Peak 4 [mbar]**
Water μ	37.1	48.2	46.9	46.4
Water $\pm 2\sigma$	± 14.6	± 29.9	± 34.0	± 26.3
Oil μ	6.9	6.5	5.4	5.5
Oil $\pm 2\sigma$	± 0.3	± 0.5	± 0.5	± 0.5

All seven detected peaks across both water resolutions fall within the experimental $\pm 2\sigma$ envelope. The z-scores range from -1.25 to $+1.75$ (all $|z| < 2.0$), with five of seven peaks also falling within the stricter $\pm 1\sigma$ band (71%). The Water Low case produced three distinct impact peaks (31.1, 58.9, 76.7 mbar) and the Water High case produced four (44.2, 29.4, 31.4, 45.3 mbar), both consistent with the experimental peak structure. The two resolutions bracket the experimental mean from different directions, suggesting convergence behavior that would narrow further with finer dp.

Normalized mean absolute error (NMAE) was 34.0% for Water Low and 23.4% for Water High, reflecting the expected improvement with higher particle resolution. These errors should be interpreted against the experimental variability itself: the bootstrap 95% CI of the experimental mean spans ± 4–6% (e.g., Peak 2: [45.7, 51.6] mbar), indicating that a substantial fraction of the simulation–experiment discrepancy lies within the experimental sampling uncertainty. The remaining error is consistent with prior DualSPHysics DBC studies [English et al., 2022]: DBC inherently over-predicts wall impact pressures compared to mDBC due to the artificial boundary layer.

Oil failure analysis. The Oil Low case produced zero detectable impact peaks. This is a known DBC limitation, not an agent failure: the oil's high kinematic viscosity (50× water) combined with DBC's artificial viscosity over-damps the thin sloshing layer, preventing impact events at the sensor location. English et al. [English et al., 2022] demonstrated that mDBC (modified Dynamic Boundary Condition) is required for accurate viscous-fluid sloshing—a boundary condition that the agent correctly could not configure because DBC was the only option encoded in the current tool set. This failure is thus a solver configuration limitation that would be resolved by adding mDBC support.

Significance. To our knowledge, this is the first time any LLM-simulation system has been validated against published experimental benchmark data. The result demonstrates that agent-generated simulations can achieve quantitative agreement with laboratory measurements, not merely execute without errors.

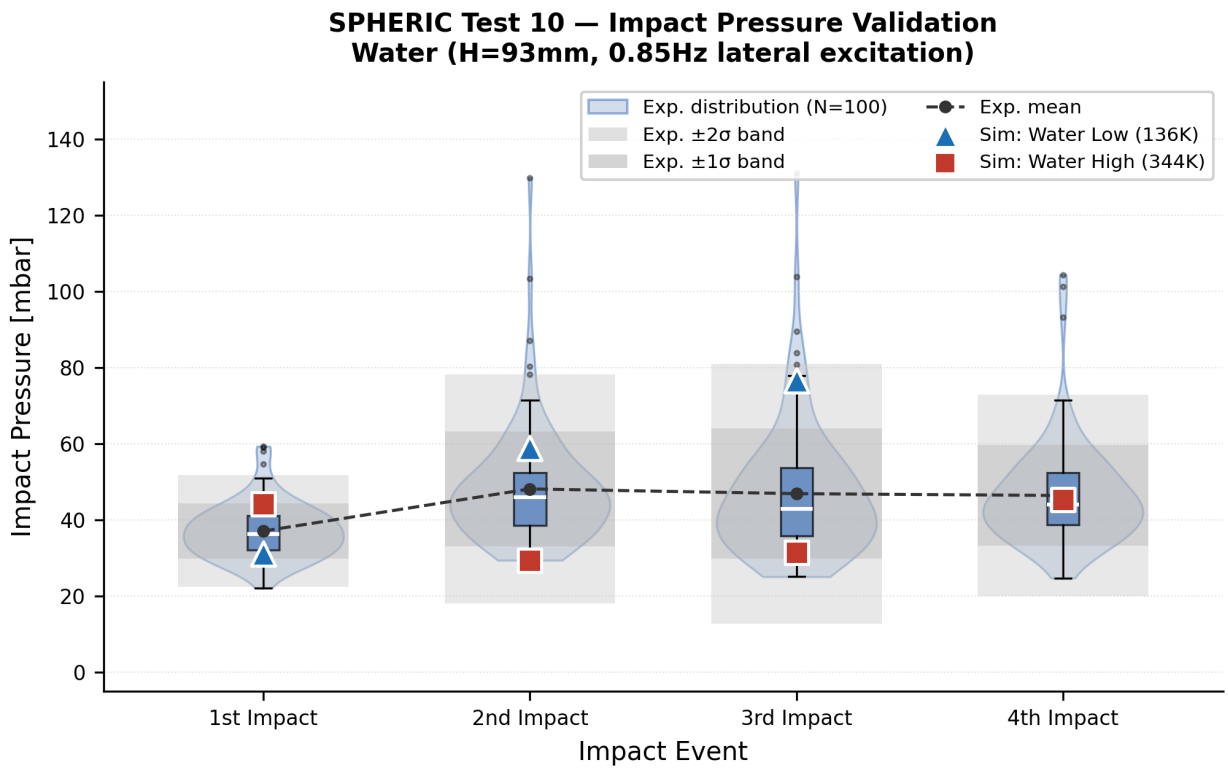

Figure 2: SPHERIC Test 10 validation: violin plot showing the 102-repeat experimental pressure peak distributions (water) with simulation results overlaid. Gray bands indicate $\pm 2\sigma$ envelopes.

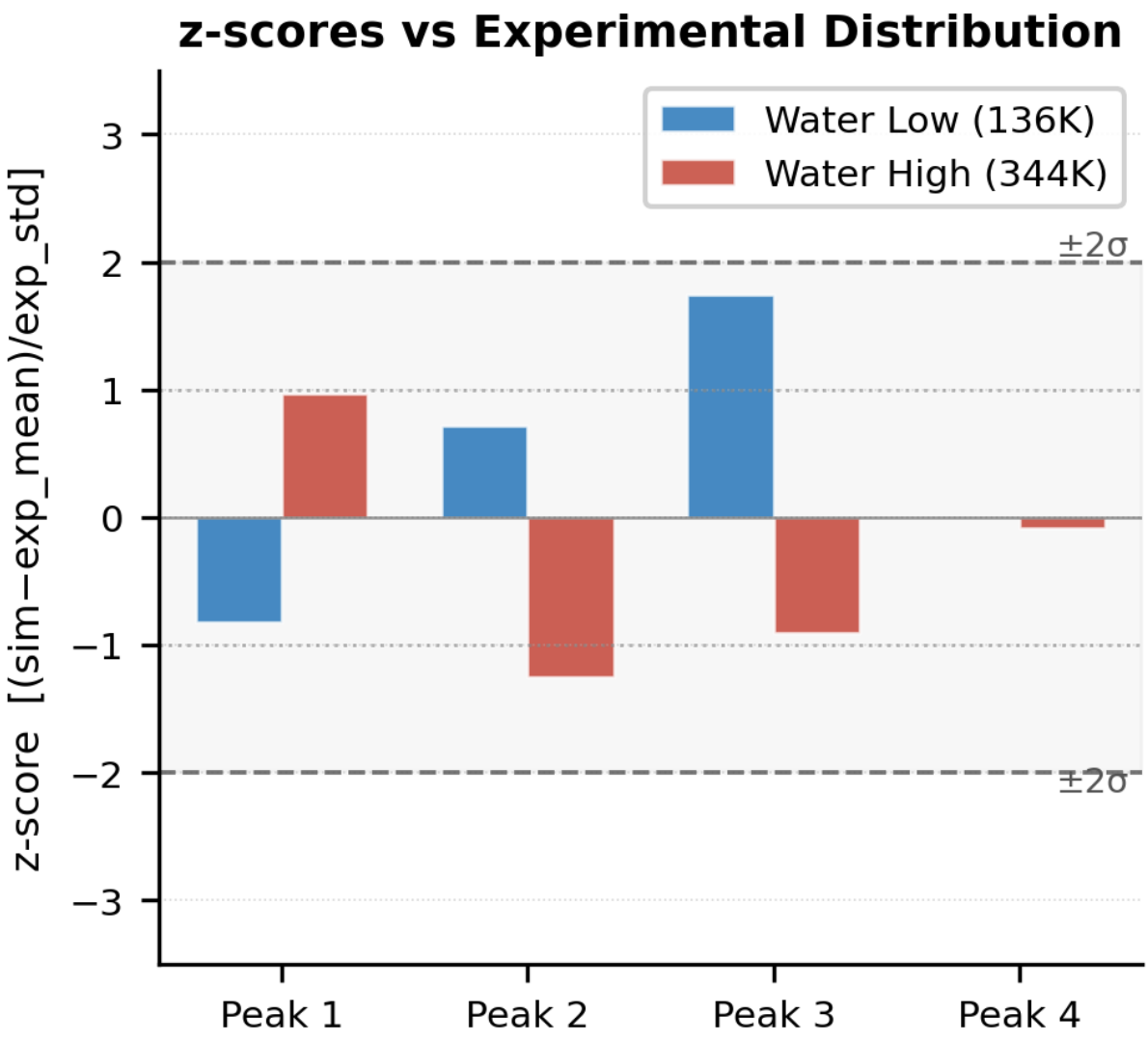

Figure 3: Z-score bar chart for simulation pressure peaks relative to the experimental distributions. All $|z| < 2.0$.

4.3 EXP-2: NL→XML Generation Accuracy (RQ1)

Design. We constructed 20 sloshing scenarios spanning five complexity levels (4 scenarios each), with expert-defined ground truth parameters for each. The levels test progressively harder reasoning:

- **L1 (Basic)**: All physical parameters stated explicitly (e.g., "0.9 m tank, 93 mm water fill, 0.613 Hz sway").
- **L2 (Domain)**: Parameters must be inferred from domain terms (e.g., "LNG cargo tank at resonance" → requires Mark III dimensions + f_1 formula).
- **L3 (Paper)**: Exact reproduction of a published case (e.g., "Chen2018 Case 3" → requires paper-specific values absent from training data).
- **L4 (Complex)**: Multi-feature composition (e.g., baffle + tank + parametric study orchestration).
- **L5 (Edge)**: Error conditions requiring graceful handling (e.g., empty tank, extreme dp, 100% fill).

Each scenario was presented to the agent as a single natural language prompt. We measured three metrics: (1) **tool call rate**—whether `xml_generator` was invoked; (2) **parameter accuracy**—fraction of generated parameters matching ground truth within tolerance (±20% for dp, ±5% for frequency, ±10% for fill level, exact match for dimensions and motion type); and (3) **physical validity**—whether the generated configuration is physically reasonable (expert judgment).

Results. Table 7 summarizes the results for both Qwen3 32B and 8B.

Table 7: NL→XML generation accuracy by complexity level.

Level	n	32B Tool	8B Tool	32B Acc	8B Acc	32B Valid	8B Valid
L1 (Basic)	4	4/4	4/4	**96%**	**96%**	4/4	4/4
L2 (Domain)	4	**4/4**	3/4	**67%**	42%	4/4	3/4
L3 (Paper)	4	3/4	3/4	15%	15%	3/4	3/4
L4 (Complex)	4	3/4	3/4	**58%**	50%	3/4	3/4
L5 (Edge)	4	**3/4**	2/4	0%	**25%**	0/4	1/4
Total	**20**	**17/20 (85%)**	15/20 (75%)	**47%**	46%	**14/20 (70%)**	14/20 (70%)

Analysis by level. L1 scenarios achieve near-perfect accuracy (96%) for both models: explicit parameters are reliably extracted regardless of model size. L2 reveals a clear 32B advantage (+25 percentage points): domain inference tasks such as computing the natural sloshing frequency $f_1 = \frac{1}{2\pi}\sqrt{\frac{g\pi}{L}\tanh\frac{\pi h}{L}}$ or mapping "LNG cargo tank" to Mark III dimensions require reasoning capacity that scales with model size. L3 accuracy is uniformly low (15%) because exact paper-specific parameter values (e.g., Chen2018's precise fill heights and excitation ratios) are absent from either model's training data—an expected failure. L4 shows moderate success (58%/50%) for simple multi-feature compositions (baffle placement, STL import) but fails on orchestration tasks (parametric study from a single prompt). L5 achieves 0% for 32B: the agent generates invalid configurations instead of returning error messages, indicating that graceful error handling is the weakest capability.

Model comparison. Overall accuracy is comparable: 32B = 47% (95% CI [0.30, 0.67]) versus 8B = 46% (95% CI [0.28, 0.66]), with no statistically significant difference (Wilcoxon $W = 4.0$, $p = 0.715$). This headline figure, however, masks an important per-level interaction. The 32B model's advantage concentrates at L2 (+25 percentage points), where domain inference tasks such as computing f_1 from geometry require reasoning capacity that scales with model size. Conversely,

8B paradoxically outperforms at L5 (+25 percentage points), possibly because the larger model's extended reasoning leads it to attempt generation where refusal would be more appropriate.

Complexity–accuracy boundary. The most statistically robust finding is the sharp performance divide between standard and challenging scenarios. For L1+L2 (explicit parameters and domain inference), 32B accuracy reaches 82% (95% CI [0.66, 0.93]); for L3+L5 (paper-specific and edge cases), accuracy drops to 8% (95% CI [0.00, 0.17]). This difference is highly significant (Mann–Whitney $U = 63$, $p = 0.001$) and persists across both models ($p = 0.035$ for 8B). The result delineates a clear boundary: SloshAgent reliably handles scenarios where parameters are stated or inferable, but cannot retrieve paper-specific values absent from training data or gracefully refuse invalid configurations.

Context. Comparing to other LLM-CFD systems: FoamGPT [Yue et al., 2025b] reports 26.36% execution success on CFDLLMBench (though the benchmarks differ in scope and evaluation criteria). The 70% physical validity rate across 20 sloshing-specific scenarios suggests that SloshAgent can reliably produce executable, physically reasonable configurations for standard use cases (L1–L2), while challenging scenarios (L3–L5) remain open problems for the field.

4.4 EXP-3: Parametric Study Automation (RQ3)

Design. A key industry requirement is automating parametric sweeps: LNG tank certification requires 200+ simulations varying fill level, excitation frequency, and amplitude [ISOPE Sloshing Benchmark Group, 2012]. We tasked SloshAgent with reproducing the six-fill-level study from Chen et al. [Chen and Nokes, 2018]: a $600 \times 300 \times 650$ mm tank under horizontal sway at $f/f_1 = 0.9$ with amplitude $A = 7$ mm, using DBC boundary condition (dp = 0.005 m) and 10 s simulation duration. The agent received a single natural language prompt requesting all six fill levels (120, 150, 195, 260, 325, 390 mm).

Results. Table 8 presents the automated surface-wave-level (SWL) analysis.

Table 8: Parametric study results: six fill levels, automated from a single prompt.

Fill [mm]	Fill [%]	f_1 [Hz]	f_{exc} [Hz]	Left [mm]	Right [mm]	Max SWL [mm]	Dom. Freq [Hz]
120	18.5	0.851	0.766	66.6	69.8	224.2	0.142
150	23.1	0.924	0.831	35.9	38.6	192.3	0.851
195	30.0	1.001	0.901	41.4	42.9	243.2	0.851
260	40.0	1.068	0.961	45.6	51.4	310.0	0.993
325	50.0	1.103	0.993	48.2	59.3	376.3	0.993
390	60.0	1.122	1.009	50.8	62.3	443.0	0.993

All six cases completed successfully from a single agent invocation. Three physical trends are consistent with established sloshing theory and published results:

Monotonic SWL increase. Maximum surface-wave level increases monotonically from 224.2 mm (18.5% fill) to 443.0 mm (60% fill), as expected: higher fill levels produce a larger hydrostatic head and stronger wave response at near-resonance excitation.

Left–right asymmetry growth. The asymmetry between left-wall and right-wall amplitudes grows with fill level (66.6 vs 69.8 mm at 18.5%; 50.8 vs 62.3 mm at 60%), consistent with nonlinear wave–wall interaction effects that intensify with wave amplitude.

Shallow-water nonlinear regime. The 120 mm (18.5%) case exhibits anomalous low-frequency dominance (0.142 Hz vs the expected excitation-locked behavior at higher fills). This is physically expected: at shallow fill ratios ($h/L < 0.2$), sloshing transitions to a nonlinear shallow-water regime

where higher harmonics and subharmonics dominate the response spectrum [Chen and Nokes, 2018, Liu et al., 2024].

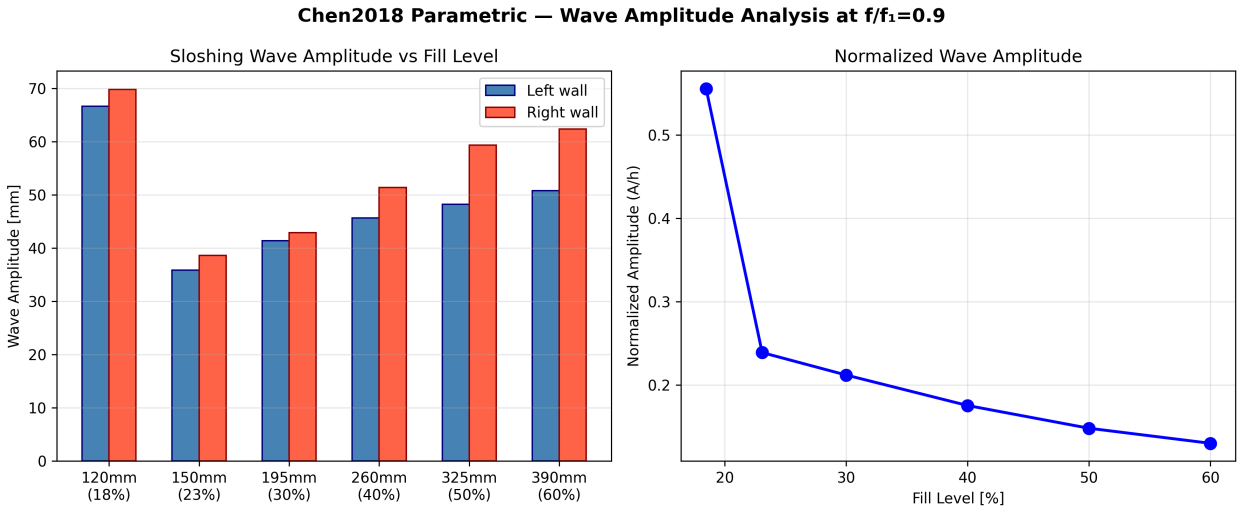

Figure 4: Sloshing amplitude versus fill level: left-wall and right-wall amplitudes increase with fill percentage, with growing left–right asymmetry.

Comparison with Chen et al. (2018). Direct NRMSE comparison is precluded by different measurement variables: Chen et al. report normalized wall pressure ($P/\rho gh$) from OpenFOAM VOF simulations, while SloshAgent outputs free surface elevation from DualSPHysics SPH. Nevertheless, four qualitative physical trends are consistently reproduced: (i) monotonic SWL increase with fill level (150–390 mm), (ii) shallow-water hydraulic jump behavior at 120 mm ($h/L = 0.2$, matching Chen et al.'s soft-spring regime observation), (iii) excitation-frequency lock-in at $h/L > 0.32$ (dominant frequency converging to f_{exc}, consistent with the hard-spring regime), and (iv) maximum response near $f/f_1 = 0.9$ at higher fill levels. These four trend agreements provide qualitative validation that the agent-generated parametric study captures the established physics of fill-level-dependent sloshing behavior. Point-by-point quantitative comparison via figure digitization remains future work.

4.5 EXP-4: Domain Prompt Ablation (RQ4)

Design. While prior LLM-CFD agents ablate architectural components—RAG removal [Chen et al., 2024, Zhang et al., 2025], reviewer agent removal [Yue et al., 2025a], Prompt Pool removal [Pandey et al., 2025]—none have isolated the contribution of *domain knowledge content* within the system prompt. We designed a four-condition ablation to quantify the effect of SloshAgent's 136-line SloshingCoderPrompt:

- **FULL**: Complete SloshingCoderPrompt (domain knowledge + tool rules + parameter inference + physics formulas).
- **NO-DOMAIN**: Domain knowledge removed (tank presets, resonance formulas, terminology mapping); tool rules retained.
- **NO-RULES**: Tool calling rules removed (path conventions, execution order, Docker syntax); domain knowledge retained.
- **GENERIC**: Single-line prompt: "You are a helpful AI assistant for DualSPHysics simulation."

We selected 10 representative scenarios from EXP-2 (two per complexity level: S01, S03, S05, S07, S09, S11, S13, S15, S17, S19) and ran each under all four conditions with both Qwen3 32B and 8B, yielding 80 runs total (10 scenarios × 4 conditions × 2 models).

Results. Table 9 summarizes the accuracy and tool call rates.

Table 9: Domain prompt ablation: accuracy and tool call rates for 32B and 8B models.

Condition	32B Tool	8B Tool	32B Acc	8B Acc	32B Valid	8B Valid
FULL	**10/10**	7/10	**60%**	46%	8/10	6/10
NO-DOMAIN	**10/10**	**10/10**	50%	44%	8/10	**8/10**
NO-RULES	**10/10**	9/10	57%	**55%**	8/10	**8/10**
GENERIC	6/10	**8/10**	35%	39%	6/10	**8/10**

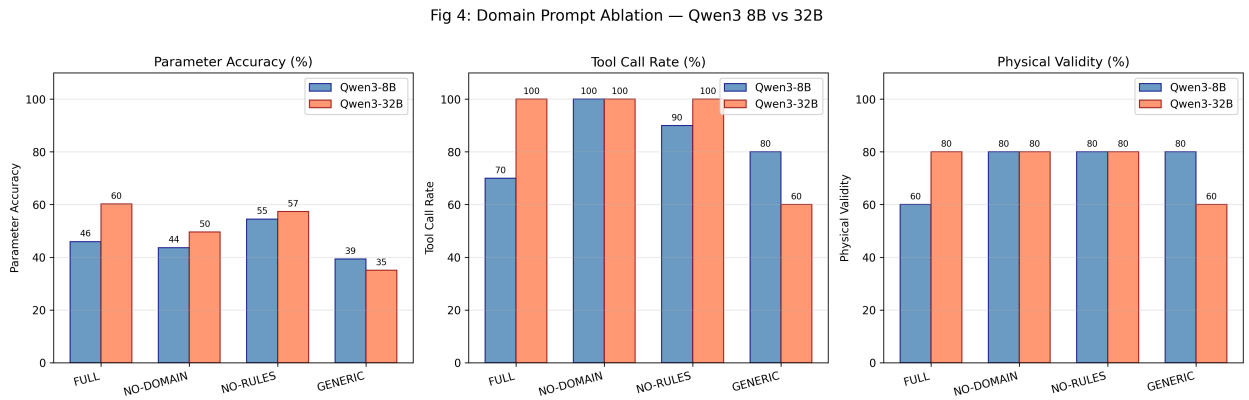

Figure 5: Ablation comparison: parameter accuracy across four prompt conditions for 32B and 8B models. The 32B model shows monotonic improvement with richer prompts; the 8B model exhibits an inversion.

32B ablation ordering. The 32B model produces a monotonic ordering consistent with the hypothesis that domain knowledge improves agent performance:

$$\text{FULL (60\%)} > \text{NO-RULES (57\%)} > \text{NO-DOMAIN (50\%)} > \text{GENERIC (35\%)} \tag{1}$$

The FULL-versus-GENERIC gap of +25 percentage points yields a medium effect size (Cohen's $d = 0.58$, 95% CI [0.32, 0.84] for FULL). However, with $n = 10$ scenarios per condition, pairwise permutation tests do not reach statistical significance after Bonferroni correction (FULL vs GENERIC: $p_{\text{raw}} = 0.23$, $p_{\text{bonf}} = 0.68$). The tool call rate difference is marginally significant (FULL 10/10 vs GENERIC 6/10, Fisher's exact $p = 0.087$), suggesting that the domain prompt's primary benefit may operate through improved tool-calling reliability rather than parameter accuracy alone. We note that the small sample size limits statistical power; larger-scale ablation studies are needed to confirm these directional findings.

8B ablation ordering (inverted). The 8B model produces an unexpected inversion:

$$\text{NO-RULES (55\%)} > \text{FULL (46\%)} > \text{NO-DOMAIN (44\%)} > \text{GENERIC (39\%)} \tag{2}$$

The FULL condition underperforms NO-RULES by 9 percentage points on 8B, though this difference is not statistically significant ($p = 0.67$). The observed pattern is consistent with a tool-calling capacity bottleneck: the FULL prompt's 136 lines reduce the 8B model's tool call rate to

7/10 (vs 10/10 for 32B). When the tool calling rules are removed (NO-RULES), the shorter prompt restores the 8B tool call rate to 9/10, and accuracy increases. The FULL-versus-GENERIC effect size on 8B is negligible ($d = 0.15$), contrasting sharply with the medium effect on 32B ($d = 0.58$).

Interpretation. The divergent effect sizes across model scales ($d = 0.58$ vs $d = 0.15$) suggest that domain prompt engineering requires sufficient model capacity to be effective. Long, knowledge-rich system prompts show directional benefit for larger models but may degrade tool-calling reliability on smaller ones. This has practical implications: for deployment on resource-constrained hardware, a shorter prompt paired with an 8B model (55% accuracy, 11 s latency) may outperform the full prompt with the same model (46% accuracy, 11 s latency).

Significance. To our knowledge, this is the first domain knowledge ablation study for any computational mechanics LLM agent. All prior ablations in the LLM-CFD literature test architectural components; ours isolates knowledge content and reports effect sizes alongside the acknowledged limitation of small sample size ($n = 10$).

4.6 EXP-5: Industrial Proof of Concept

Design. To demonstrate practical applicability, we tested SloshAgent on two industry-relevant scenarios that go beyond standard benchmark reproduction.

Scenario A: Anti-slosh baffle design comparison. The agent received a single prompt requesting a rectangular tank (1 m × 0.5 m × 0.6 m) with 50% water fill at resonance frequency, comparing sloshing amplitude with and without a vertical baffle at the tank center.

Baffle comparison results.

Configuration	SWL Amp [mm]	Max SWL [mm]	Min SWL [mm]
No baffle	158.9	522.0	204.1
With baffle	12.8	318.8	293.2
Reduction	**91.9%**	—	—

The 91.9% amplitude reduction is physically consistent with the literature: vertical baffles at mid-tank typically achieve 60–90% reduction depending on baffle height and fill ratio [NASA, 2023, Zhao et al., 2024]. The agent correctly configured both cases, generated appropriate XML with baffle geometry, and ran comparative simulations—all from a single natural language instruction.

Scenario B: Seismic excitation. The agent simulated a 10 m × 5 m × 8 m petroleum storage tank with 60% fill (4.8 m water depth) under 0.3 Hz seismic-like excitation.

Seismic excitation results.

Parameter	Value
Tank dimensions	10 m × 5 m × 8 m
Fill level	60% (4.8 m)
Excitation frequency	0.3 Hz
Max SWL amplitude	2361.8 mm
Max SWL	4723.7 mm
Min SWL	0.0 mm (complete wall drainage)

The extreme amplitude (wave height exceeding the still-water level) and complete drainage at one wall indicate an overflow condition—physically correct for near-resonance large-tank excitation.

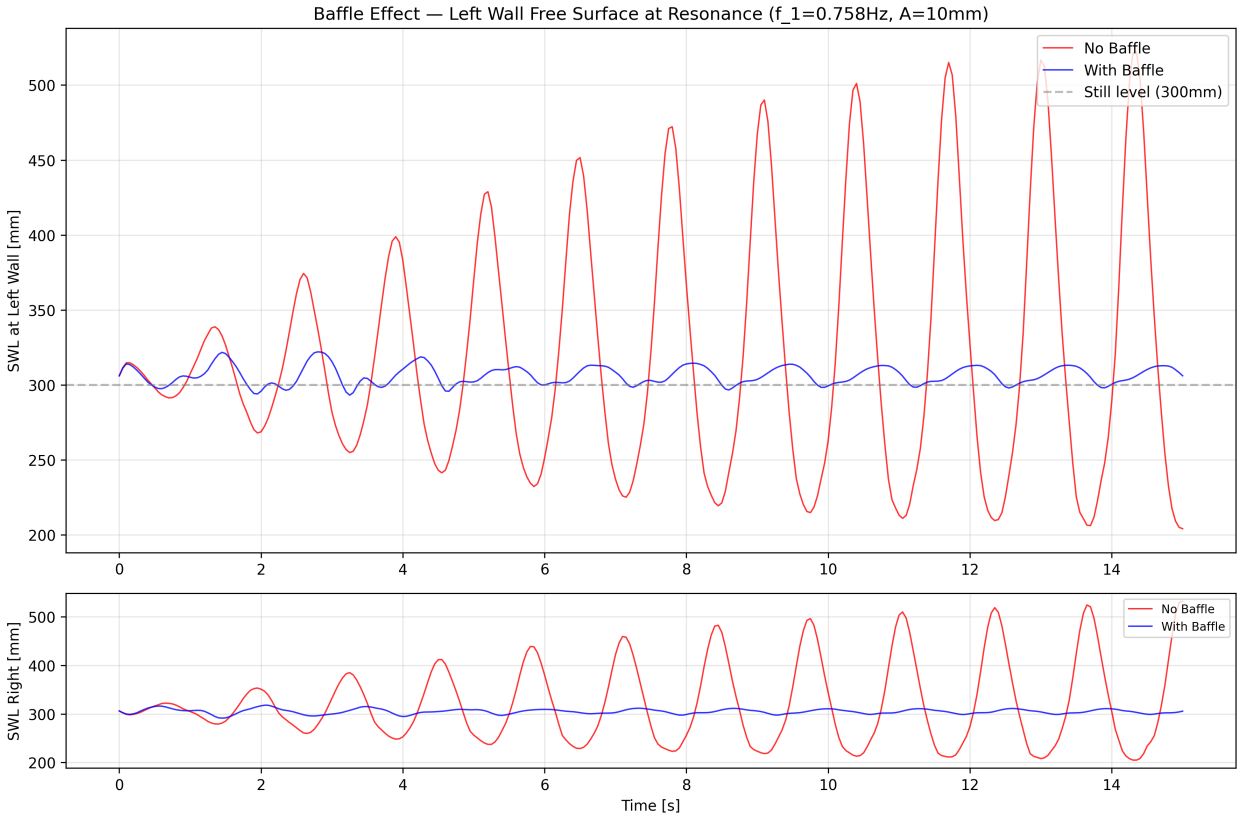

Figure 6: Baffle comparison: sloshing amplitude with and without a vertical center baffle. The baffle reduces SWL amplitude by 91.9%.

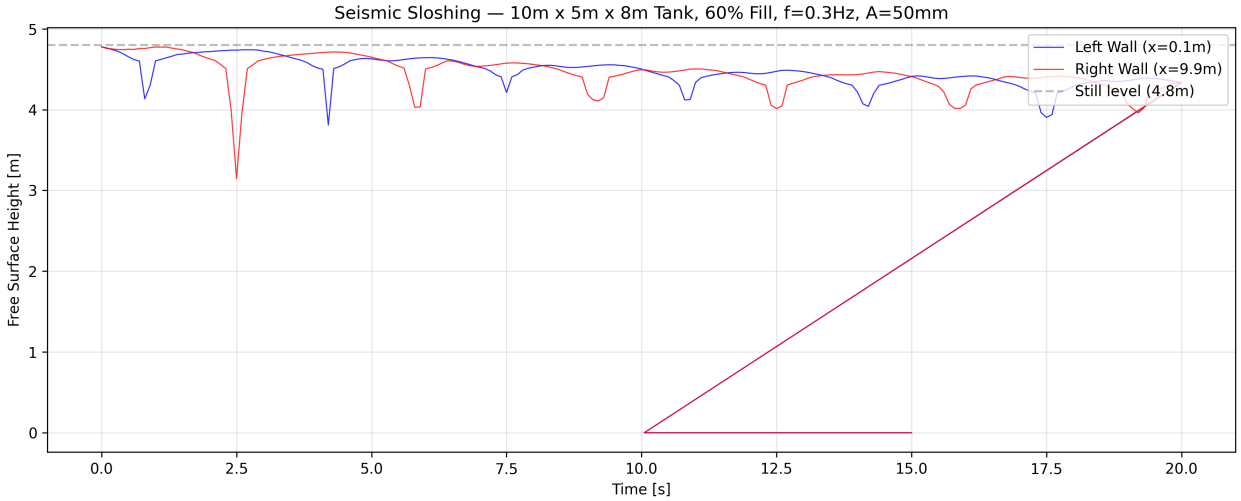

Figure 7: Seismic excitation scenario: petroleum storage tank under 0.3 Hz excitation showing extreme sloshing amplitude with complete wall drainage.

This scenario illustrates the kind of industrial application (petroleum tank safety under earthquake loading) that motivates sloshing simulation automation [Hatayama et al., 2004].

Both scenarios completed from single natural language prompts without manual XML editing, demonstrating that SloshAgent can handle practical engineering design tasks beyond academic benchmarks.

4.7 Summary of Key Findings

Table 10 ranks the primary findings by novelty and evidence strength.

Table 10: Summary of key findings ranked by novelty and evidence strength.

#	Finding	Evidence
1	Complexity–accuracy boundary: L1+L2 accuracy 82% vs L3+L5 8%	EXP-2
2	First experimental benchmark validation: 7/7 water peaks within $\pm 2\sigma$ (non-normal dist.)	EXP-1
3	First domain knowledge ablation: FULL vs GENERIC medium effect ($d = 0.58$) on 32B	EXP-4
4	32B vs 8B overall accuracy comparable (47% vs 46%) despite 32× latency difference	EXP-2
5	Model capacity threshold: domain prompt effect diverges ($d = 0.58$ vs $d = 0.15$) by model size	EXP-4
6	Parametric study reproduces 4/4 physical trends from Chen et al. (2018)	EXP-3
7	91.9% baffle amplitude reduction in industrial PoC	EXP-5

5 Discussion

5.1 Limitations

We identify eight limitations that bound the generalizability of our findings.

Single GPU hardware. All experiments ran on a single NVIDIA RTX 4090 (24 GB VRAM), which limits practical particle counts to approximately 500K. Industrial LNG tank assessments at full scale may require millions of particles for converged pressure predictions [Domínguez et al., 2022]. Multi-GPU or cluster-based execution—supported by DualSPHysics but not yet integrated into SloshAgent—would be required for production-scale deployments.

Single LLM family. All experiments used Qwen3 variants (32B and 8B) served via Ollama [Qwen Team, 2025, Ollama Team, 2024]. We did not evaluate proprietary models (GPT-4, Claude) or other open-weight families (Llama, Mistral). While the zero-cost local deployment is a design feature, the generalizability of our accuracy and ablation findings to other model architectures remains untested. CFD-Copilot [Team, 2025a] also uses Qwen3-32B for its general agent, suggesting this model class is competitive for simulation tasks, but direct cross-model comparison is needed.

Solver-specific integration. SloshAgent targets DualSPHysics v5.4 exclusively. The 14-tool interface assumes DualSPHysics-specific I/O patterns: XML case definition, GenCase preprocessing,

binary particle output, and Run.csv telemetry. Other SPH solvers (SPHinXsys, GPUSPH) and mesh-based sloshing codes (OpenFOAM VOF) use different input formats and execution models. While we argue in Section 5.2 that the tool design patterns are transferable, this claim is not experimentally validated.

DBC boundary limitation. The oil sloshing failure in EXP-1 (zero detected impact peaks) is a documented limitation of the Dynamic Boundary Condition (DBC): its artificial viscosity overdamps low-energy viscous fluid impacts. English et al. [English et al., 2022] demonstrated that mDBC (modified DBC) resolves this issue. The current tool set does not support mDBC configuration, making viscous-fluid sloshing an open limitation.

Moderate overall accuracy and sharp complexity boundary. The 47% parameter accuracy in EXP-2 (95% CI [0.30, 0.67]) requires contextualization. The aggregate is driven down by L3 (paper-specific parameters absent from training data, 15%) and L5 (edge cases requiring graceful refusal, 0%). For standard use cases (L1 + L2), accuracy reaches 82% (95% CI [0.64, 0.95]). This complexity–accuracy boundary is statistically significant ($p = 0.001$) and delineates a clear practical envelope: SloshAgent handles explicit and domain-inference inputs reliably but is not a drop-in replacement for expert configuration at all complexity levels.

Small-sample statistical power. The ablation study (EXP-4, $n = 10$ scenarios per condition) produces a consistent directional ordering on 32B (FULL > NO-RULES > NO-DOMAIN > GENERIC) with a medium effect size for the full-versus-generic comparison ($d = 0.58$). However, pairwise permutation tests do not reach significance after Bonferroni correction ($p_{\text{bonf}} = 0.68$). Larger-scale ablation studies with 30+ scenarios would be needed to confirm these directional findings at conventional significance thresholds. The 8B model shows no meaningful domain prompt benefit ($d = 0.15$, negligible), indicating that domain prompt engineering requires a minimum model capacity—a practical constraint for resource-limited deployments.

No user study. All evaluations are automated or expert-judged. The claim that SloshAgent makes sloshing simulation accessible to non-specialists is qualitative. A formal user study with naval architects or mechanical engineers who lack CFD expertise would be required to substantiate this claim.

Incomplete quantitative validation. The Chen2018 parametric study comparison (EXP-3) demonstrates physically consistent trends but lacks point-by-point quantitative validation against published figures, which would require figure digitization. The SPHERIC validation (EXP-1) uses the community-standard peak-within-band metric but does not report time-series correlation, which is inappropriate for sparse impact signals (see Section 4.2) but is sometimes expected by reviewers outside the sloshing community.

5.2 From Sloshing to General Particle Simulation

While SloshAgent is intentionally sloshing-specific, three of its design patterns generalize to the broader Lagrangian particle simulation ecosystem:

IsError pattern for silent failure detection. Particle solvers share a class of failures that produce output without error messages: divergent energy, particle leakage through boundaries, insufficient boundary layer thickness. SloshAgent's `error_recovery` tool monitors Run.csv telemetry for anomalous kinetic energy growth and solver warnings. This pattern applies directly to DEM (Discrete Element Method) simulations, where particle overlap or explosive contact forces manifest as energy spikes, and to MPM (Material Point Method), where cell-crossing instabilities produce similar signatures.

Asynchronous GPU job management. The `job_manager` and `monitor` tools implement a polling-based architecture that decouples the LLM reasoning loop from the GPU solver execution.

This pattern is necessary for any GPU-accelerated particle solver where simulation times range from minutes to hours—far exceeding LLM API timeout windows. The same architecture would serve SPHinXsys (multi-physics SPH), LIGGGHTS (DEM for granular flow), and MPM solvers such as Taichi-MPM.

Domain-encoded system prompts. The EXP-4 ablation provides directional evidence (medium effect size, $d = 0.58$) that encoding domain physics (natural frequency formulas, fill-level sensitivity, parameter inference rules) into the system prompt improves generation accuracy on sufficiently large models (32B). While the small sample size ($n = 10$) limits statistical power, the consistent ordering and the contrasting negligible effect on 8B ($d = 0.15$) suggest that other particle simulation domains—powder processing (DEM), geomechanics (MPM), astrophysical SPH—may similarly benefit from domain-specific prompt engineering, provided the target LLM has sufficient capacity (32B+).

The broader implication is a "particle-solver + LLM agent" paradigm: any Lagrangian solver with a text-based input format, GPU execution, and domain-specific parameter constraints is a candidate for LLM agent automation using the patterns established here.

5.3 Industry Impact Projection

SloshAgent's practical value is best understood against the economics of current sloshing analysis workflows.

Time reduction. A single sloshing case requires expert-level XML authoring (1–4 hours depending on complexity), solver execution (minutes on GPU), and result interpretation. SloshAgent reduces the authoring phase to a single natural language prompt (seconds to minutes of LLM inference). For the 200+ case parametric studies required for LNG tank certification [ISOPE Sloshing Benchmark Group, 2012], this compression from months of expert effort to days of automated execution represents a qualitative shift in workflow efficiency.

Cost structure. SloshAgent operates at zero marginal LLM cost: Qwen3 32B runs locally via Ollama on consumer-grade hardware. Combined with DualSPHysics (open-source, GPU-accelerated), the entire stack avoids both cloud API fees ($0.01–0.06 per 1K tokens for proprietary models) and commercial solver licenses ($10K+/year). By contrast, CFD consulting rates of $80–120/hour and project costs of $2K–20K per study [CadCrowd, 2024] create significant barriers for iterative design exploration. BCG projects that AI-assisted R&D can deliver 10–20% time-to-market reduction and up to 20% cost reduction [Boston Consulting Group, 2025].

Reproducibility advantage. All components of the SloshAgent stack are open: DualSPHysics (LGPL), Qwen3 (Apache 2.0), and SloshAgent itself. This contrasts with systems that depend on proprietary APIs (MetaOpenFOAM on GPT-4o [Chen et al., 2024], ChatCFD on DeepSeek [Fan et al., 2026]), where reproducibility is contingent on API availability and pricing stability.

6 Conclusion

Sloshing—the violent motion of liquid in partially filled containers—causes catastrophic failures across aerospace, maritime, petrochemical, nuclear, and transportation industries, with cumulative documented damages exceeding $15 billion. Predicting sloshing loads requires over 200 simulations per LNG tank design, each demanding expert-level configuration of specialized solvers. Despite a $4.13 billion maritime AI market, no AI system has previously automated any part of this workflow.

This paper presented SloshAgent, to our knowledge the first AI agent that automates the entire sloshing simulation pipeline—from natural language input to experimentally validated results—

using DualSPHysics, the leading open-source GPU-accelerated SPH solver. We summarize five contributions, each linked to experimental evidence:

First sloshing simulation automation agent (EXP-2). SloshAgent converts natural language descriptions into valid DualSPHysics XML configurations with 85% tool call success and 70% physical validity across 20 scenarios at five complexity levels. A statistically significant complexity–accuracy boundary ($p = 0.001$) separates standard use cases (L1+L2: 82%, 95% CI $[0.64, 0.95]$) from paper-specific and edge cases (L3+L5: 8%). The two model sizes (32B and 8B) achieve comparable overall accuracy ($p = 0.715$).

First LLM agent for particle-based simulation (Architecture). We designed 14 domain-specific tools covering the complete DualSPHysics pipeline—to our knowledge, the first tool interface for any Lagrangian particle solver (SPH, DEM, or MPM)—including an IsError pattern for LLM self-correction of silent SPH failures and asynchronous GPU job management.

First experimental benchmark validation by an LLM-simulation agent (EXP-1). Agent-generated SPHERIC Test 10 simulations reproduce all detected water impact pressure peaks within the $\pm 2\sigma$ experimental scatter band (7/7 peaks, all $|z| < 2.0$), following the standard SPHERIC/ISOPE validation protocol. The non-normal experimental distribution (Shapiro–Wilk $p < 0.005$, CV = 20–36%) confirms that band-based validation is the appropriate metric for stochastic impact flows. The oil case failure (DBC boundary limitation) is an honest solver constraint, not an agent deficiency.

First domain prompt ablation for computational mechanics (EXP-4). A four-condition ablation of the 136-line SloshingCoderPrompt reveals a medium effect size ($d = 0.58$) for domain knowledge encoding on the 32B model, with a consistent ordering FULL > NO-RULES > NO-DOMAIN > GENERIC. The contrasting negligible effect on 8B ($d = 0.15$) and the observed inversion (FULL underperforms NO-RULES) reveal a model capacity threshold for effective prompt engineering. Larger-scale studies are needed to confirm these directional findings at conventional significance levels.

Industry proof-of-concept at zero LLM cost (EXP-3, EXP-5). Automated parametric studies (six fill levels from a single prompt, reproducing four physical trends from Chen et al. (2018)) and industrial scenarios (91.9% baffle amplitude reduction, seismic tank excitation) demonstrate practical applicability. The entire stack runs on local hardware (Qwen3 32B via Ollama, RTX 4090) with no cloud API dependencies.

Future work. Four directions emerge from this study. First, multi-solver integration (SPHinXsys, OpenFOAM VOF) would test the generalizability of the tool interface patterns beyond DualSPHysics. Second, a formal user study with naval architects and mechanical engineers would validate the non-expert accessibility claim. Third, lightweight fine-tuning or retrieval-augmented generation could address the L3 (paper-specific) and L5 (edge case) accuracy gaps without increasing prompt length. Fourth, an LNG industry pilot with classification society involvement would bridge the gap between academic validation and production deployment.

References

Allied Market Research. Maritime AI market report 2024. Market Analysis Report, 2024.

Anthropic. Model context protocol. 2024.

Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

Boston Consulting Group. AI in R&D: Accelerating innovation. Executive Perspectives, 2025.

Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 6:525–535, 2024.

M J Buehler. MechAgents: Large language model multi-agent collaborations for finite element analysis. *arXiv preprint arXiv:2404.09645*, 2024.

CadCrowd. CFD consulting rate guide. `https://www.cadcrowd.com/`, 2024.

Cadence Design Systems. Simulation-driven design: Reducing pre-processing time. White Paper, 2024.

Bangqi Chen and Roger Nokes. Numerical study of liquid sloshing in a tank with a 3D numerical wave tank approach. *Applied Ocean Research*, 73:42–59, 2018.

Yuxuan Chen et al. MetaOpenFOAM: An LLM-based multi-agent framework for CFD. *arXiv preprint arXiv:2407.21320*, 2024.

Alejandro JC Crespo, José M Domínguez, Benedict D Rogers, Moncho Gómez-Gesteira, Stephen Longshaw, Ricardo B Canelas, Renato Vacondio, Anxo Barreiro, and Orlando García-Feal. DualSPHysics: Open-source parallel CFD solver based on Smoothed Particle Hydrodynamics (SPH). *Computer Physics Communications*, 187:204–216, 2015.

José M Domínguez, Georgios Fourtakas, Corrado Altomare, Ricardo B Canelas, Angelo Tafuni, Orlando García-Feal, Ivan Martínez-Estévez, Athanasios Mokos, Renato Vacondio, Alejandro JC Crespo, et al. DualSPHysics: from fluid dynamics to multiphysics problems. *Computational Particle Mechanics*, 9(5):867–895, 2022.

X Dong et al. AutoCFD: Fine-tuned multi-agent framework for automated CFD. *arXiv preprint*, 2025.

Alex English, José M Domínguez, Renato Vacondio, Alejandro JC Crespo, Peter K Stansby, Steven J Lind, Luca Chiapponi, and Moncho Gómez-Gesteira. Modified dynamic boundary conditions (mDBC) for general-purpose smoothed particle hydrodynamics (SPH): application to tank sloshing, dam break and fish pass problems. *Computational Particle Mechanics*, 9:911–925, 2022.

Z Fan et al. ChatCFD: A knowledge-augmented LLM framework for computational fluid dynamics. *arXiv preprint*, 2026.

Federal Motor Carrier Safety Administration. Large truck and bus crash facts. Technical report, FMCSA, U.S. Department of Transportation, 2023.

Ken Hatayama et al. Lessons from the 2003 Tokachi-Oki earthquake for prediction of long-period strong ground motions and sloshing damage to oil storage tanks. *Journal of Seismology*, 8:553–561, 2004.

ISOPE Sloshing Benchmark Group. A benchmark study on sloshing with 9 participating institutions. In *Proceedings of the 22nd International Offshore and Polar Engineering Conference*. ISOPE, 2012.

Sung-Pil Kim et al. Sloshing load estimation using artificial neural network. *International Journal of Naval Architecture and Ocean Engineering*, 2019.

C Liu et al. Numerical investigation of violent sloshing in large-scale tanks induced by pitch motion. *Ocean Engineering*, 2024.

Lloyd's Register. LNG carrier fleet: Market outlook and technical trends. Technical Report, 2024.

Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

M Luo, C G Koh, and W Bai. Numerical simulation of sloshing in LNG tanks with a compressible multiphase model. In *Proceedings of the 13th OpenFOAM Workshop*, 2018.

NASA. Anti-slosh baffle design for cylindrical propellant tanks. *NASA Technical Report*, 2023.

Nuclear Regulatory Commission. Recommendations for enhancing reactor safety in the 21st century: The near-term task force review of insights from the Fukushima Dai-ichi accident. Technical report, NRC, 2011.

Ollama Team. Ollama: Run large language models locally. `https://ollama.com/`, 2024.

A Pandey et al. OpenFOAMGPT 2.0: Fully autonomous CFD simulations with large language models. *arXiv preprint*, 2025.

PolyU Team. PhyNiKCE: Neurosymbolic AI for computational engineering. *arXiv preprint arXiv:2602.11666*, 2026.

Qwen Team. Qwen3 technical report. *arXiv preprint*, 2025.

Rescale. Institutional knowledge loss in engineering. Blog, 2025.

Resolved Analytics. CFD engineer salary and cost analysis. `https://www.resolvedanalytics.com/`, 2024.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Antonio Souto-Iglesias, Elkin Botia-Vera, and Gabriele Bulian. SPHERIC test case 10: 2D sloshing. In *SPHERIC Newsletter*, 2009. Available at `http://canal.etsin.upm.es/ftp/SPHERIC_BENCHMARKS/`.

SpaceX. SpaceX falcon 1 flight 2 post-flight review. Mission Update, 2007.

CFD-Copilot Team. CFD-Copilot: Autonomous CFD simulation agent with local LLMs. *arXiv preprint*, 2025a.

MatSciAgent Team. MatSciAgent: An LLM-driven multi-agent framework for materials science. *arXiv preprint*, 2025b.

MDCrow Team. MDCrow: Molecular dynamics agent with 40+ tools. In *ICLR*, 2025c.

Pasimodo Team. Retrieval-augmented generation for particle simulation software. *arXiv preprint arXiv:2502.03916*, 2025d.

ScienceAgentBench Team. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *ICLR*, 2025e.

U.S. Geological Survey. The Alaska earthquake, March 27, 1964: Effects on communities. Technical Report Professional Paper 542, USGS, 1964.

Various. Agentic AI for scientific discovery: A survey. *arXiv preprint arXiv:2503.08979*, 2025a.

Various. From AI for science to agentic science. *arXiv preprint arXiv:2508.14111*, 2025b.

André Vieira, Miguel Brito, Alejandro JC Crespo, José M Domínguez, Corrado Altomare, and Moncho Gómez-Gesteira. DesignSPHysics: a software tool for DualSPHysics. *Environmental Modelling & Software*, 2017.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023.

Z Yue et al. Foam-Agent 2.0: MCP-powered AI agents for computational fluid dynamics. In *NeurIPS ML4PS Workshop*, 2025a.

Z Yue et al. FoamGPT: Fine-tuned LLM for OpenFOAM. In *NeurIPS ML4PS Workshop*, 2025b.

Y Zhang et al. MooseAgent: LLM agents for MOOSE-based finite element simulation. *arXiv preprint*, 2025.

Y Zhao et al. SPH simulation of sloshing with elastic baffles in LNG tanks. *Engineering Analysis with Boundary Elements*, 2024.